

VITyarthi BYOP: Project Statement

STATEMENT.MD

Course: Programming in Java (CSE2006) | Section 5.2 Deliverable

Project Title:	Hangman Game in Java	Student Name:	Jainam Jain
Course Code:	CSE2006 (Programming in Java)	Registration No:	25BAI11474

Project Document: `statement.md`

1. Problem Statement

Many basic terminal and console games developed during introductory programming courses suffer from core structural and usability defects:

- **Monolithic Single-File Anti-Pattern:** All game routines, dictionary lists, and input handlers are compressed into a single procedural file, making extension and debugging difficult.
- **Fragile Input Handling:** Erroneous user inputs (symbols, numbers, empty returns) cause crashes (e.g., parsing exceptions) or penalize the player unfair attempt deductions. Duplicate guesses also deduct attempts rather than alerting the player.
- **Volatile Session Lifetime:** Player scores and achievements disappear as soon as the console window closes, providing no persistent progress or leaderboard tracking.

This project resolves these issues by decomposing the software into dedicated Object-Oriented classes, introducing defensive input validation routines, and integrating local file storage to permanently log player outcomes.

2. Scope of the Project

In-Scope Deliverables

- Console application split into functional OOP classes.
- 3 difficulty modes (Easy: 7, Medium: 5, Hard: 4 attempts).
- Dynamic string masking with underscore representation.
- Defensive sanitization without penalty on duplicate/bad input.
- Persistent local storage using `scores.txt` via File I/O.
- Automated logic verification via `TestGame.java`.

Out-of-Scope Constraints

- Heavy desktop graphical interfaces (JavaFX / Swing).
- Multiplayer networked socket communication.
- External relational database management systems (RDBMS).
- Cloud-based online leaderboard APIs.

3. Target Users

Students & Programming Learners

Individuals seeking clean, documented examples of Java OOP design, class modularity, defensive programming, and simple file I/O operations.

Casual Console Gamers

Players looking for an engaging, crash-resistant, terminal-based word puzzle that tracks scores across multiple play sessions.

4. High-Level Features

- **Modular Architecture:** Decoupled modules (`Game`, `WordList`, `Validator`, `Player`, `ScoreBoard`) ensuring clean separation of concerns.
- **Defensive Validation Layer:** Sanitizes keystrokes to ensure alphabetic entry and blocks duplicate guesses with zero penalty deduction.
- **Persistent File Management:** Automatically computes session scores ($\text{Attempts Left} \times 10$) and reads/appends records to `scores.txt`.
- **Automated Test Rig:** A standalone validation class (`TestGame.java`) executing unit checks on masking, turn decrements, and input rules.